

Banking System

SE – 4111 Course Project

Submitted to: Dr. Mawal A. Mohammed

Prepared by:

S	ID	Name
1	445050303	Abdulaziz Albarqawi
2	445050085	Mohammed Aljuhayyim
3	444050349	Rayan Saad
4	445050229	Bassam Saleh

Nov. 2026

Table of Contents

1. Requirements……4
2. Design……6
3. Implementation……..8
4. Lesson Learned……..……..10

Executive Summary

This report describes the Banking System we built for SE-4111. It is a small desktop application where a user selects an account, checks its balance, deposits and withdraws money, transfers funds to another account, and views the transaction history. We developed it in Java using Swing and kept the scope deliberately small: two sample accounts, in-memory data storage, and no login or database. This allowed us to focus on the quality of the design and the correctness of the banking operations.

The design follows the Model-View-Controller (MVC) pattern. BankModel manages the data and enforces the banking rules, BankView displays the information, and BankController connects user actions to the model. We also used the Observer pattern to keep the interface synchronized with the data. After each successful operation, the model notifies the view, which updates the displayed balance and transaction history. Account numbers are represented by an immutable AccountId class, documented with a representation invariant and an abstraction function. The class also implements matching equals() and hashCode() methods to ensure correct behavior when used as a map key. Money is stored using BigDecimal to avoid rounding errors.

The rest of this report presents the system requirements, explains the design decisions and implementation, includes screenshots of the application, and discusses the key lessons learned during development.

1. Requirements

1.1 Application Concept

The proposed application is a desktop-based Banking System designed to manage basic banking operations for customers and their accounts. The system allows users to view account information, deposit and withdraw money, transfer funds between accounts, and review transaction history. The application maintains the current account balance and transaction records as its internal state, which changes according to user actions.

The system follows the Model-View-Controller (MVC) architecture. The Model manages customer, account, balance, and transaction data. The Controller handles user actions and applies the required banking operations while validating input and preventing invalid transactions. The View provides a graphical interface that displays account information, balances, and transaction history and updates the displayed information whenever the application state changes.

1.2 Functional Requirements

FR-1: Account Information

The system shall allow the user to view the selected bank account's information, including account number, account holder name, and current balance.

FR-2: Deposit Money

The system shall allow the user to deposit a specified amount into the selected account and update the account balance accordingly.

FR-3: Withdraw Money

The system shall allow the user to withdraw a specified amount from the selected account if the account has sufficient funds.

FR-4: Fund Transfer

The system shall allow the user to transfer a specified amount from one account to another, provided that the source account has sufficient funds.

FR-5: Transaction History

The system shall record and display the transaction history of an account, including the transaction type, amount, date, and resulting balance.

FR-6: Account Balance Update

The system shall automatically update and display the account balance after every successful deposit, withdrawal, or transfer operation.

2. Design

Technical and design requirements	Description of integration into your application
Architectural Pattern	The application is structured using MVC: • Model (BankModel): Owns the account holders, balances and transaction records, and checks the banking rules itself. Because the rules live here, an operation is validated the same way whether it comes from the GUI or from a test with no GUI at all. The model knows nothing about Swing. • View (BankView): The main window. It shows the selected account's number, holder and balance, the action controls and the transaction history. It registers with the model and refreshes itself when the model reports a change. • Controller (BankController): Handles the button clicks. It reads what the user typed, checks that the amount is a valid number, calls the right method on the model, and then shows either a success message or an error. BankApp creates the three objects and connects them.
Design Patterns	We used the Observer pattern. BankModel is the "Subject" and BankView is the "Observer": the view implements the BankModel.Listener interface and registers itself with the model. **Justification:** The balance and the history must update after every successful operation (FR-6). The model only knows about the Listener interface, so it does not depend on Swing or on any particular window. After a successful deposit, withdrawal or transfer the model calls modelChanged(), and the view reads the new balance and history and redraws. A rejected request changes nothing and sends no notification.
Abstract Data Types (ADTs)	In addition to the classes needed to get the app working, the primary ADT is: AccountId: (immutable) Represents a bank account number. Its abstract value is exactly six decimal digits. Leading zeros matter, so 100001 identifies one account. The representation is private final String value.

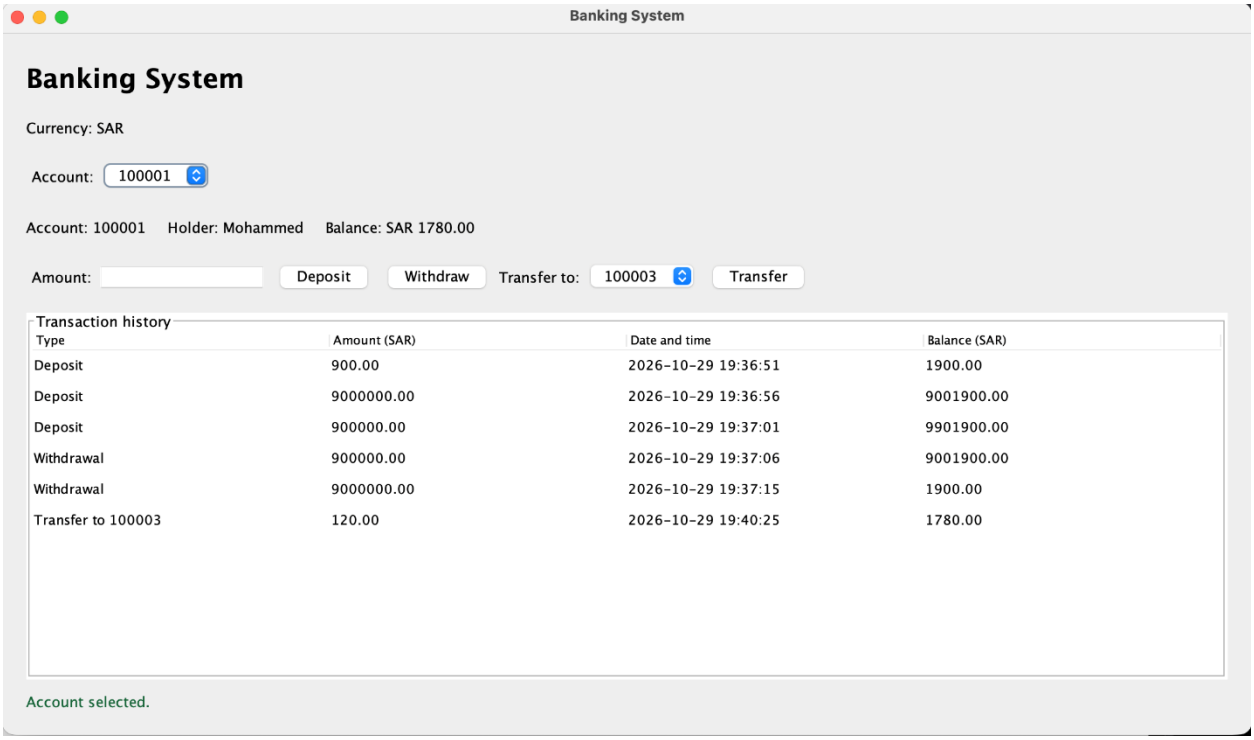
Technical and design requirements	Description of integration into your application

	`//RI:`
	`//value != null`
	`//value matches [0-9]{6}`
	`//  (exactly six decimal digits)`
	
	`//AF`
	`//AF(value) = the bank account identifier`
	`//represented by the six characters in value`
	
	Operations: AccountId(String) creates a valid identifier or throws IllegalArgumentException. value() and toString() return the six digits.
	
	Why it is immutable: the class is final, the field is private and final, String is itself immutable, and no method changes the identifier. Returning the String therefore does not expose any mutable state. The RI applies to each identifier on its own; making sure two accounts never share a number is the job of the model's maps.

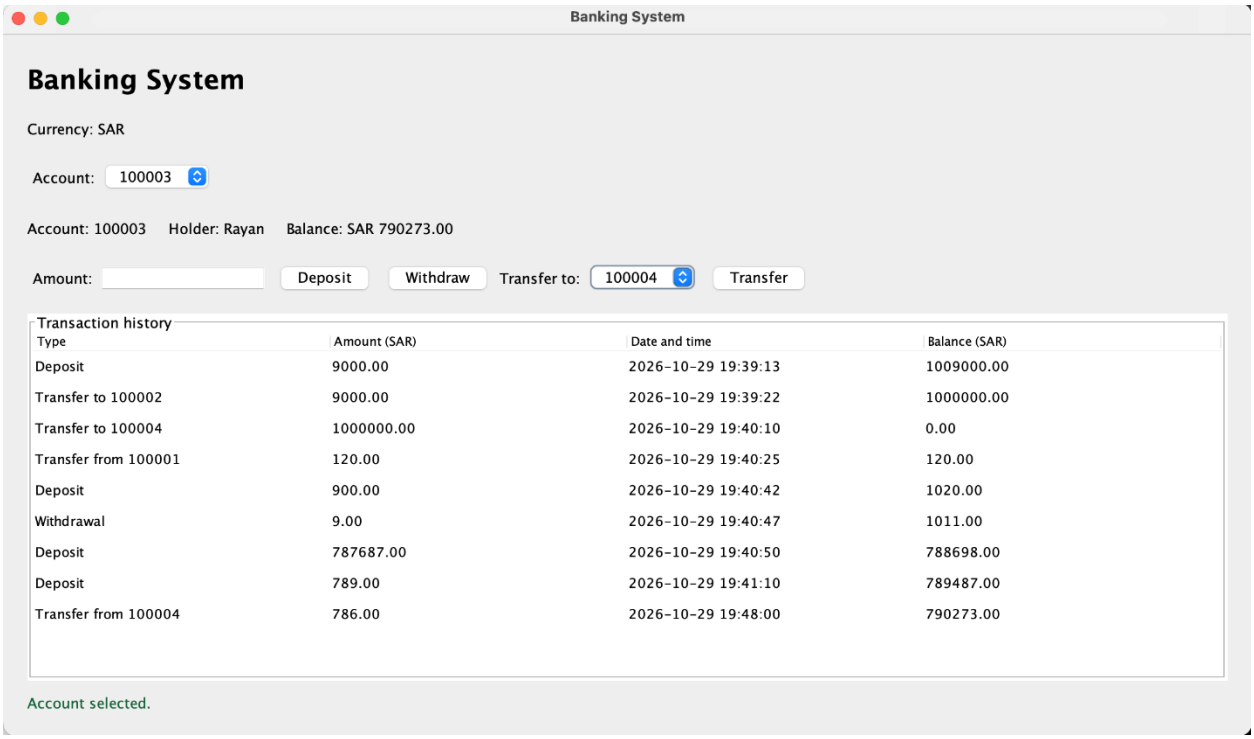
Equality	The equals() and hashCode() methods are implemented for the AccountId ADT, both based on the account-number value. Two AccountId objects holding 100001 are equal and have the same hash code, and comparing with null returns false. **Justification:** AccountId is the key in the model's maps. Without these methods,balance(new AccountId("100001")) would fail to find an account that was created with a different AccountId object holding the same number. Because equality compares immutable String values, it is reflexive, symmetric, transitive and consistent.

3. Implementation

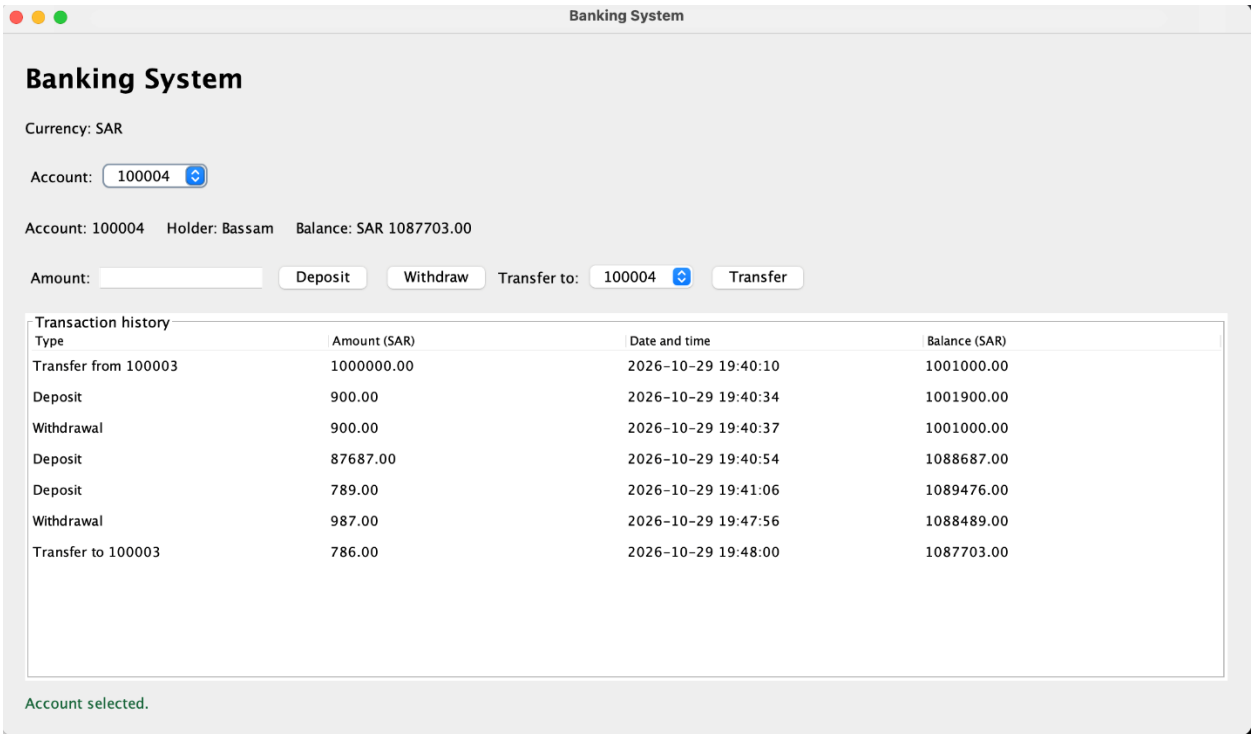
3.1 Account information (FR-1)



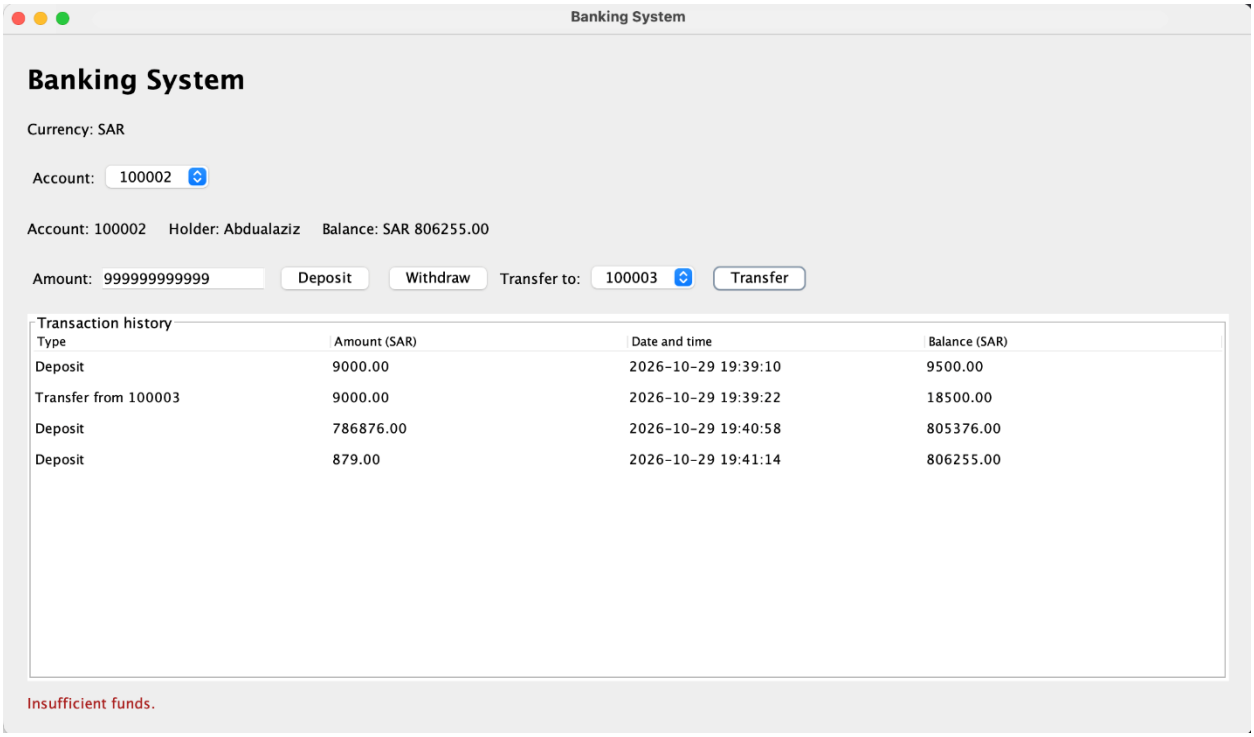
3.2 Successful banking operations (FR-2 to FR-6)



3.3 Destination account after the transfer



3.4 Invalid operation handling



3.5 GitHub source code

[GitHub Link]

4. Lessons learned

4.1 Separating responsibilities

This project showed us why MVC is worth the effort. With the banking rules in the Model, we could check them without opening the interface at all. The Controller only connects the buttons to those rules, and the View only displays what it is given. That means we can change the screen layout without touching the deposit or withdrawal logic.

4.2 Keeping the screen consistent

Observer was the simplest way we found to keep the display in step with the data. After a successful operation the Model sends one notification and the View refreshes the balance and history. For a transfer, the timing matters: the Model notifies only after both accounts have been updated, so the screen never shows a half-finished transfer.

4.3 Protecting data and defining equality

Designing AccountId taught us that an immutable class takes more than a private field. It also needs validation in the constructor, a final representation and no methods that change the value. Writing the RI and AF forced us to say exactly what an account number is, which made the class clearer. Implementing equals() and hashCode() together showed us why two separate objects for the same account must behave as the same key in a map.

4.4 Testing failures as well as successes

Banking operations made us think about bad input and boundary cases: zero or negative amounts, rounding money by accident, and withdrawing more than the balance. A transfer has to validate its destination before it touches the source account. We found that testing that a failed operation leaves both balances and both histories unchanged matters just as much as testing a successful one.

4.5 Improvements for a later version

The current version meets all six functional requirements using sample accounts and in-memory storage. A later version could add persistent storage and more accounts. If we added a database or background operations, we would also need proper transaction handling and a clear concurrency policy. For this project, keeping the scope small made the design easier to understand and to verify.